class06_AO

Ashley Osuna

Q1: Write a function add() to add some numbers together. Make sure to add comment(s) to the function explaining why you do things.Your first version of the function should add two input numbers together; e.g. add(4, 7) should return 11

```
add_2<-function(x,y){
  x+y}
#adding variables x and y.
#added the add_2 function to assign
#make sure the function is adding both
#variables or knows to add both variables
```

Answer Q1.a.: The function of addition is proven by adding 4 and 7 together below.

```
add_2(4,7)
```

```
[1] 11
```

Q1.b.:Your second version of the function should add any numbers together that the user inputs; e.g. add(1, 2, 3, -4) should return 2.

Answer Q1.b.: Below we used add_multi2 with sum added as a function to add multiple numbers together

```
add_multi2<-function(x,
...){sum(x,...)}
#Using sum to add multiple numbers
add_multi2(1,2,3,-4)
```

```
[1] 2
```

Q2 : Write a function generate_dna() that generates a random DNA sequence of a length input by the user; e.g. generate_dna(11) may return "ATTTAAGGCTG". Make sure to add comments to the function. Play with functions sample() and paste() in your R-Studio Console to see how they can help here. Answer Q2: We used dna_function() as well as included nucleotides to then have the nucleotides as a vector.This aided in making the vector into a string by using paste()

```
generate_dna<-function(x){
  #generating a nucleotide vector
nucs<-c("A","G","T","C")
#using sample to pull out random nucleotides
nucs_vector<-sample(nucs,size=x,
                    replace=TRUE)
#generating a dna string from nucs_vector
 paste(nucs_vector,collapse = "")}
#testing my generate_dna() function
generate_dna(10)
```

```
[1] "GTCGCCCCTG"
```

```
#make vector into a string by using paste
```

Q3: Write a function generate_protein() that generates a random peptide/protein sequence of a length input by the user; e.g. generate_protein(6) may return "WQRTAG". Make sure to use the single-letter code for all 20 amino acids and use no other letters (see list of amino acids at the bottom of the page); make sure that individual amino acids can appear more than once. Make sure to add comments to the function Answer Q3:We added all 20 amino acids to make a protein vector then generated the protein vector into a string.

```
#assigning aa to be recognized as the list of the 20 amino acids
generate_protein<-function(x){aa<-c("A",
  "R","N","D","C","L","K",
  "M","F","P","S","T","W",
  "Y","V","Q","E","H","G",
  "I")
  #adding a protein vector of the sample
 protein_vector<-sample(aa,size=x,
 replace=TRUE)
 #generating a protein string from protein_vector
```

```
 paste(protein_vector,collapse="")}
 #checking/texting my generate_protein() to give 6 random amino acids
 generate_protein(6)
```

```
[1] "VWEMMA"
```

Q4: Use your generate_protein() function to generate peptides of lengths 5, 7, 9, and 11 amino acids. Take advantage of the base R function sapply() for this. Answer Q4: sapply() was added on the code below to avoid calling the function four seperate times. Thus, this function adds efficiency

```
#defining the lengths desired
lengths<-c(5,7,9,11)
#applying the function to each length (runs
#the function multiple times accross a vector)
sapply(lengths,generate_protein)
```

```
[1] "RPWIV"       "ERGYPIM"     "ESAQVLEVL"   "AYEITGSMFRR"
```

Q5: Take each of your peptides from Q4 and run a BLASTP search selecting the Non-redundant protein sequences (nr) database and homo sapiens (taxid: 9606) organism. For each search, before clicking BLAST check the Show results in a new window tab next to the BLAST button to easily allow you to run the blast search for all four peptides at the same time in four different tabs. make sure to type out your calculations in your Quarto document. Upload your numbers for each peptide (length (aa) and max identity (%)) into the shared class excel data sheet (see Data Sheet link in the Canvas assignment).Answer Q5: Each sequence was calculated below with the following codes and their outputs were the calculations being done by R.**

```
#first  sequence of length of 5 = LAAFS. Percent
#identity 100% and Query cover is 100%
1.0*1.0
```

```
[1] 1
```

```
#second  sequence of length of 7 =
#WMLNTRP with Percent identity 46.67% and
#Query cover is 100%
0.4667*1
```

```
[1] 0.4667
```

```
#third  sequence of length of 9 = SKQMQVCHV
#with Percent identity 75% and Query
#cover is 89%

0.75*0.89
```

```
[1] 0.6675
```

```
#fourth/last  sequence of length of 11 =
#TYSHPELFFFL with Percent identity
#72.73% and Query cover is 91%

0.7273*0.91
```

```
[1] 0.661843
```

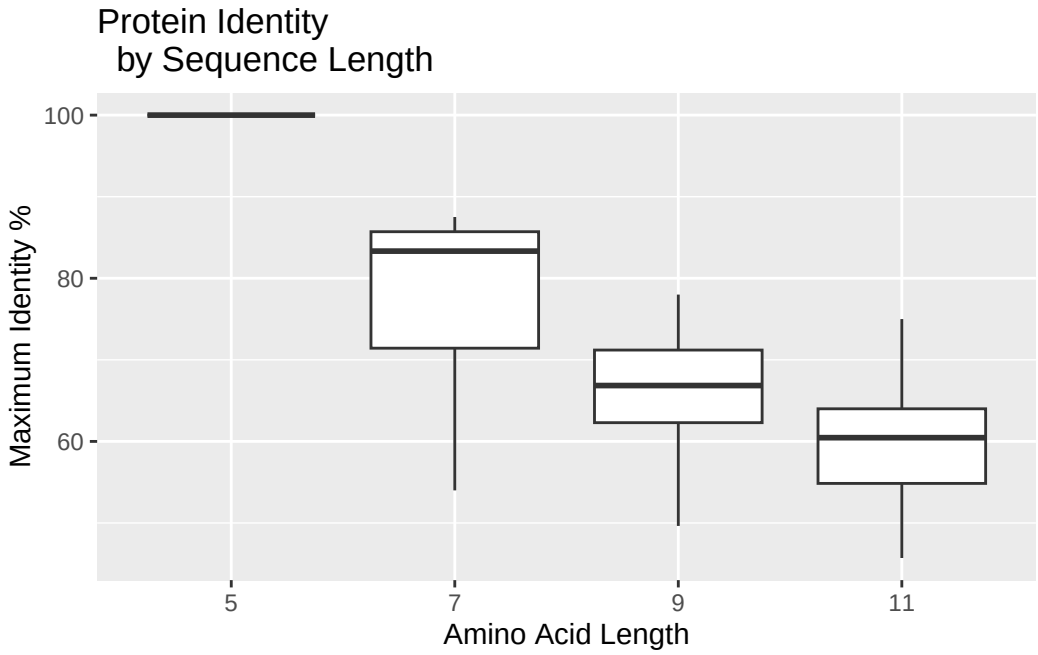
Q6: upload the csv file into a data frame using the command peptide_df <- read.csv("your_filename"). Use the head() function to list the first 10 rows Answer Q6: we inouted read.csv for the data to be recognized to then use the data in the future (for the following questions)

```
#inputing read.csv function for the data to be read
peptide_df<-read.csv("BIMM143_peptide(Sheet1).csv")
#testing data to be read
head(peptide_df,n=10)
```

```
   Length_aa Max_Identity_perc Your_Initials
1          5            100.00           JLA
2          7             85.71           JLA
3          9             66.85           JLA
4         11             55.00           JLA
5          5             80.00           SKB
6          7             85.71           SKB
7          9             66.85           SKB
8         11             64.00           SKB
9          5            100.00           KET
10         7             85.71           KET
```

Q7:Generate boxplot Answer Q7: Boxplot was generated by first making sure library(ggplot2) was downloaded. Then, generated box plot with ggplot set up (aes and titles for x,y axis, and geom_boxplot).Also, exlcuded outliers to be counted for in geom_boxplot.

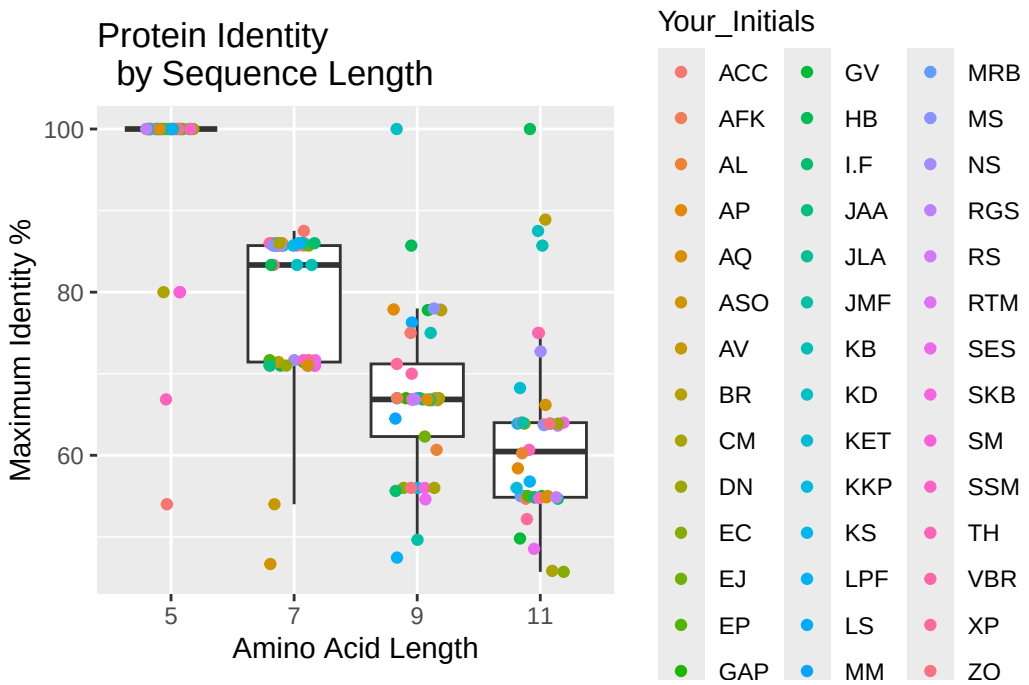
```
#load in library
library(ggplot2)

#generating a box plot with appropriate titles
p<-ggplot(peptide_df)+
  aes(x=as.factor(Length_aa),
  y=Max_Identity_perc)+
  geom_boxplot(outlier.shape=NA)+
  labs(title="Protein Identity
  by Sequence Length",
  x="Amino Acid Length",
  y="Maximum Identity %")
#testing for plot to be shown
p
```



Q8.a.: add a geom_jitter() layer to your box plot with each data point colored according to students' initials by adding the following additional

line to your plot Answer Q8.a.: geom_jitter added and the results was color coordinating the data points per column Your_Initials

```
#referencing Your_Initials to had color coordination
p+geom_jitter(aes(col=Your_Initials),width=0.2)
```



Q8.b:First, use the rep() function to generate a vector consisting of as many repeats of "GREY" as there are datapoints in the peptide_df data frame. Next, use the Your_Initials column in the peptide_df dataframe (which can be called as peptide_df$Your_Initials) to generate a TRUE/FALSE vector indicating which datapoints are yours (TRUE) and which are not (FALSE). Now, use the mydata vector to replace the "GREY" values in the color vector that correspond to your data points with your favorite color Answer Q8.b.:I used dim() to get total number of rows in the data, and i always tested the table to ensure console tells grey with the amount. Next, I made a vector of TRUE vs FALSE data to ensure my personal data is accounted for. Finally, I added color to my personal data (mydata) with a testing following it to ensure I did it correctly.

```
#part i : using dim() to get total number of rows
color<-rep("GREY",dim(peptide_df)[1])
```

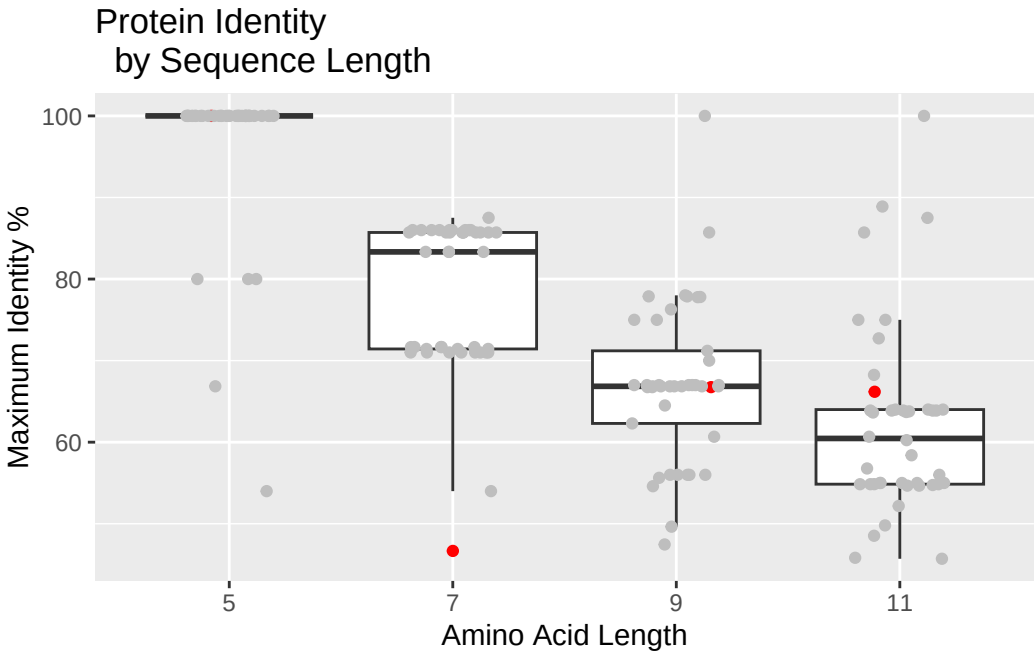
```
#testing with table() to ensure it the
#correct length, console tells grey to be 168
table(color)
```

```
color
GREY
 168
```

```
#part ii : making a vector of TRUE
#(my personal data ASO) and FALSE
#(not mydata)
mydata<-peptide_df$Your_Initials=="ASO"

#partiii : adding color to my personal
#data (ASO)
color[mydata]<-"red"
#testing the plot, console tells
#grey 164, red 4
table(color)
```

```
color
GREY  red
 164    4
```

```
#part iv : adding jitter pints
#using my red vector
p+geom_jitter(col=color,width=0.2)
```



Q8 c: Based on your boxplot, does any of your own peptide data fall outside the 25 to 75 percentile of the class's data? If yes, list which one(s) Answer Q8 c: Yes, my 7-amino acid peptide fell below the 25 to 75 percentile (became an outlier), my 11-amino acid peptide fell slightly above the median, but 11-amino acid, 9-amino acid, and 5-amino acid are within the 25 to 75 percentile.

Q9: MHC class II molecules of our immune system display 9 amino acid peptides on the surface of immune cells that, when from a foreign origin, activates a T-cell immune response. Based on your findings in Q7 and 8, speculate why those peptides are 9 amino acids long rather than, say, 5 amino acids Answer Q9 : Based on the vast majority having 100% identity for their 5-amino acid, this shows that this specific sequence is found in every protein (estimate). If the immune system used 5-amino acid, it would continuously get FALSE due to not being able to tell the difference between a dangerous virus vs its own healthy cells